

Markov Chain

Transition Probability

This is the **memoryless property**: the future depends only on the present, not the past.

$$\mathbb{P}[\text{ future } \mid \text{ present and past }] = \mathbb{P}[\text{ future } \mid \text{ present }].$$

The n -step transition matrix **Transition Probabilities** For $i, j \in S$,

$$p_{ij} = P(X_{n+1} = j | X_n = i).$$

the probability that X goes from i to j in one step (conditional on starting at i)

Example (Weather)

$$X_n = \begin{cases} 0 & \text{if dry on day } n, \\ 1 & \text{if wet on day } n. \end{cases}$$

Suppose X is a Markov chain with just two states $d = 0$ and $w = 1$, with transition probabilities $\mathbb{P}[\text{wet tomorrow} \mid \text{wet today}] = 0.8$, $\mathbb{P}[\text{dry tomorrow} \mid \text{dry today}] = 0.7$.

$$P = \begin{pmatrix} \mathbb{P}(d \rightarrow d) & \mathbb{P}(d \rightarrow w) \\ \mathbb{P}(w \rightarrow d) & \mathbb{P}(w \rightarrow w) \end{pmatrix} = \begin{pmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{pmatrix}$$

$$P(X_2 = 2 | X_0 = 1) = P_{1,2}^2$$

$$P^2 = \begin{pmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{pmatrix} \begin{pmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{pmatrix} = \begin{pmatrix} 0.55 & 0.45 \\ 0.30 & 0.70 \end{pmatrix}$$

$$P_{1,2}^2 = 0.45$$

The Chapman-Kolmogorov equations

Suppose X is a Markov chain. Then, for all $m, n \geq 0$,

$$P^{(n+m)} = P^{(n)} P^{(m)},$$

in other word,

$$p_{ij}^{(n+m)} = \sum_{k \in S} p_{ik}^{(n)} p_{kj}^{(m)}.$$

Probability being in state j after $n + m$ steps, starting from state i .

Classification of States

Accessible

For a Markov Chain $\{X_n, n = 0, 1, 2, \dots\}$ with transition probability matrix $\mathbf{P}$, state j is said to be **accessible** from state i , denoted by $i \rightarrow j$, if $P_{ij}^{(m)} > 0$ for some $m \geq 0$.

- The meaning of $P_{ij}^{(m)}$ (use $-$ to denote one transition)
 - $m = 1$: $P_{ij} > 0$ means it is possible to jump from i to j directly
 - $m = 2$: $P_{ij}^{(2)} > 0$ means there is at least one path $i - k - j$, with non-zero probability
 - $m = 3$: $P_{ij}^{(3)} > 0$ means there is at least one path $i - k - l - j$, with non-zero probability
- It holds for at least one $m > 0$, meaning there exists a path from i to j , no matter how long it is.

Communicate

If two states i and j are accessible from each other, i.e., $i \rightarrow j$ and $j \rightarrow i$, then they are said to **communicate**, denoted by $i \leftrightarrow j$.

Same communicating class i and j are in the same communicating class if either $i = j$ or $i \leftrightarrow j$, then $i \sim j$.

Communication Class

Communication is an equivalence relation, i.e.,

- Reflexivity**: $i \leftrightarrow i$.
- Symmetry**: if $i \leftrightarrow j$, then $j \leftrightarrow i$.
- Transitivity**: if $i \leftrightarrow j$ and $j \leftrightarrow k$, then $i \leftrightarrow k$.

Recurrent and Transient States

A state i is said to be **recurrent** if $f_{ii} = 1$; and **transient** if $f_{ii} < 1$.

State i is recurrent if and only if

$$\sum_{n=1}^{\infty} P_{ii}^{(n)} = \infty.$$

- If a state i is recurrent and $i \leftrightarrow j$, then state j is recurrent;
- If a state i is transient and $i \leftrightarrow j$, then state j is transient.

Absorbing

Absorbing state: if a state i satisfies that $p_{ij} = 0$ for all $j \neq i$, then it is called an "absorbing state"

Periodic

The state is revisited only in multiples of some fixed number of steps.

Long-Run Behaviour

Stationary Distribution

A vector $\pi = (\pi_j)$ is stationary if

$$\pi = \pi P, \sum_j \pi_j = 1.$$

Consider a (stationary) MC with one-step transition probability matrix

$$\mathbf{P} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{pmatrix} 0.1 & 0.5 & 0 & 0.4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix} \end{matrix}.$$

To find the limiting distribution, we set up the equation system

$$\begin{cases} \pi_0 & = & 0.1\pi_0 + \pi_3 \\ \pi_1 & = & 0.5\pi_0 \\ \pi_2 & = & \pi_1 \\ \pi_3 & = & 0.4\pi_0 + \pi_2 \\ \pi_0 + \pi_1 + \pi_2 + \pi_3 & = & 1 \end{cases}$$

Solve the equations. With the first 4 equations, we can find

$$\pi_0 = \pi_0, \pi_1 = 0.5\pi_0, \pi_2 = 0.5\pi_0, \pi_3 = 0.9\pi_0.$$

Combine with the last equation, the solution is (.3448, .1724, .1724, .3103)

```
# State order: [N, R, L]
P = np.array([
    [0.6, 0.3, 0.1], # N -> (N, R, L)
    [0.0, 0.7, 0.3], # R -> (N, R, L)
    [0.0, 0.0, 1.0]]) # L -> (N, R, L)

P2= np.linalg.matrix_power(P, 2) # or P @ P
p_NL_2= P2[0, 2] # row 0= N, col 2= L
print(f"Probability of N L after 2months: {p_NL_2:.3f}")
# Initial state distribution (all customers start as new)
pi0= np.array([1, 0, 0])
# One-step evolution
pi1= pi0 @ P
customers = 1000* pi1 # Expected number of customers in each state
print("State probabilities after one month:", pi1)
# steady-state distribution
A= P.T- np.eye(3)
A[-1, :]= 1
b= np.zeros(3); b[-1]= 1
pi= np.linalg.solve(A, b)
print(np.round(pi, 2))
```

```
states = [0, 1, 2] or #["N", "R", "L"]
# Pick a starting state at random
current_state = np.random.choice(states)
U = np.random.rand() # Draw U ~ Uniform(0,1)
# Compute cumulative distribution for the row
cum_probs = np.cumsum(P[current_state])
next_state = np.searchsorted(cum_probs, U) # Determine next state
```

```
def mc_simulate(current_state=0, steps=10):
    path = [current_state]
    for _ in range(steps):
        U = np.random.rand()
        # cumulative transition probabilities for current state
        cum_probs = np.cumsum(P[current_state])
        # choose next state
        next_state = np.searchsorted(cum_probs, U)
        path.append(next_state)
        current_state = next_state
    return [states[i] for i in path] # convert to state labels
```

Modelling Inputs, Outputs & Validation

Simulation Inputs

Sampling Plan: Determine the appropriate sample size (n) and sampling method. To estimate a population mean μ with margin of error E at confidence level $(1 - \alpha)$, the required sample size is

$$n = \left(\frac{z_{\alpha/2} \sigma}{E} \right)^2,$$

Example (Service Times):

Suppose the true standard deviation of service times is about 4 minutes. To estimate the mean within ± 1 minute with 95% confidence:

$$n = \left(\frac{1.96 \times 4}{1} \right)^2 \approx 62.$$

Probability Distribution Fitting

Parametric Distributions

Assume $X \sim f(x; \theta)$ with parameter vector θ .

Probability Distributions Summary

Distribution	Type	PMF / PDF	Mean	Variance
Bernoulli(p)	Discrete	$P(X = 1) = p$	p	$p(1 - p)$
Binomial(n, p)	Discrete	$\binom{n}{x} p^x (1 - p)^{n-x}$	np	$np(1 - p)$
Poisson(λ)	Discrete	$\frac{e^{-\lambda} \lambda^x}{x!}$	λ	λ
Geometric(p)	Discrete	$p(1 - p)^{n-1}$	$1/p$	$(1 - p)/p^2$
Uniform(a, b)	Continuous	$1/(b - a)$	$(a + b)/2$	$(b - a)^2/12$
Normal(μ, σ^2)	Continuous	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	μ	σ^2
Exponential(λ)	Continuous	$\lambda e^{-\lambda x}$	$1/\lambda$	$1/\lambda^2$

Weibull (α, β)

$$f(x) = \frac{\alpha}{\beta} \left(\frac{x}{\beta} \right)^{\alpha-1} \exp \left[- \left(\frac{x}{\beta} \right)^{\alpha} \right], \quad x \geq 0.$$

MLE

MLE is one of the most fundamental methods for estimating the parameters of a parametric distribution.

Given a dataset $x_1, x_2, \dots, x_n$, and a distribution with density or mass function $f(x; \theta)$, the likelihood function is

$$L(\theta \mid x_1, \dots, x_n) = \prod_{i=1}^n f(x_i; \theta).$$

MLE chooses the parameter value $\hat{\theta}$ that maximizes this likelihood. Equivalently, since the logarithm is increasing, $\hat{\theta} = \arg \max_{\theta} \ell(\theta)$, $\ell(\theta) = \ln L(\theta \mid x_1, \dots, x_n)$.

MLEs usually have good statistical properties (consistency, asymptotic normality), and they often lead to closed-form formulas for common distributions.

Example: MLE for the Exponential Distribution

Assume interarrival times follow

$$X \sim \text{Exponential}(\lambda), \quad f(x; \lambda) = \lambda e^{-\lambda x}, \quad x \geq 0.$$

Data (minutes):

2.1, 1.3, 0.7, 3.2, 1.9.

$$L(\lambda) = \prod_{i=1}^n \lambda e^{-\lambda x_i} = \lambda^n e^{-\lambda \sum_{i=1}^n x_i}.$$

$$\ell(\lambda) = \ln L(\lambda) = n \ln \lambda - \lambda \sum_{i=1}^n x_i.$$

$$\frac{d\ell}{d\lambda} = \frac{n}{\lambda} - \sum_{i=1}^n x_i = 0.$$

Solve for λ :

$$\hat{\lambda} = \frac{n}{\sum_{i=1}^n x_i}.$$

$$\sum_{i=1}^5 x_i = 2.1 + 1.3 + 0.7 + 3.2 + 1.9 = 9.2.$$

Thus,

$$\hat{\lambda} = \frac{5}{9.2} \approx 0.5435.$$

Non-parametric Approaches

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{X_i \leq x\}}.$$

Common Theoretical Distributions

Several distributions are frequently encountered in simulation modeling:

Chi-Square test

Hypotheses:

- H_0 : The data follows the specified distribution.
- H_1 : The data does not follow the specified distribution.

Procedure

- Divide the data into k non-overlapping bins. Discrete data: each value can be a category. Continuous data: choose bins so that each expected count is at least 5.
- Compute the observed frequency O_i for each bin i .
- Compute the expected frequency under the hypothesized distribution:

$$E_i = n P(X \in \text{bin } i),$$

where n is the sample size.

- Compute the chi-square statistic:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}.$$

- Determine degrees of freedom:

$$df = \begin{cases} k - 1, & \text{no parameters estimated,} \\ k - 1 - m, & \text{m parameters estimated.} \end{cases}$$

- Compare to the chi-square critical value. Reject H_0 if

$$\chi^2 > \chi_{\alpha, df}^2.$$

Example. Consider a random quantity that can take any of the possible values 1, 2, 3, 4, 5 and suppose we want to test the hypothesis that these values are equally likely to occur. A sample of size 50 yielded the observed frequencies

Discrete Case $N_i : 12, 5, 19, 7, 7.$ $H_0 : p_i = \frac{1}{5}.$ Since $n = 50$ and has probability $1/5$, $E_i = n \cdot \frac{1}{5} = 50 \cdot \frac{1}{5} = 10$ for all $i = 1, \dots, 5$. $\chi^2 = \sum_{i=1}^5 \frac{(N_i - E_i)^2}{E_i} = 12.8.$ $df = k - 1 = 5 - 1 = 4.$ The critical value at $\alpha = 0.05$ is $\chi_{0.05, 4}^2 = 9.488.$ Since $12.8 > 9.488$, we reject the null hypothesis.	$H_0 : X \sim \text{Exponential}(\lambda).$ $\hat{\lambda} = \frac{1}{\bar{X}} = 0.266$ The exponential CDF is $F(x) = 1 - e^{-\hat{\lambda}x}.$ The expected probabilities are $p_1 = F(2) - F(0) = 0.4139,$ $p_2 = F(4) - F(2) = 0.1700,$ $p_3 = F(6) - F(4) = 0.1356,$ $p_4 = F(8) - F(6) = 0.0770,$ $p_5 = 1 - F(8) = 0.1314.$ Thus the expected frequencies are $E_i = p_i \cdot n$ $\chi^2 = \sum_{i=1}^5 \frac{(O_i - E_i)^2}{E_i} = 4.378$ The critical value at $\alpha = 0.05$ is $\chi_{0.05, 3}^2 = 7.815.$ Since $4.378 < 7.815$ Fail to reject null hypothesis
--	---

```
system_size = []
time_points = []

def customer(env, cid, teller, mu):
    """Customer process: arrival, request, wait, get served."""
    # --- Arrival ---
    arrival_time = env.now
    print(f"C{cid} arrives at {arrival_time:.2f}")

    # Record system size BEFORE joining queue
    # N = queue length + # being served + this arriving customer
    system_size.append(len(teller.queue) + teller.count + 1)
    time_points.append(env.now)
    # --- Request server ---
    with teller.request() as req:
        yield req # pausing until server is free
    # --- Waiting time ---
    wait = env.now - arrival_time
    print(f"C{cid} starts at {env.now:.2f}, waited {wait:.2f}")
    # --- Service time ---
    service_time = np.random.exponential(1 / mu)
    yield env.timeout(service_time)
    print(f"C{cid} finishes at {env.now:.2f}")

    # --- Server is released here, queue size may decrease ---
    system_size.append(len(teller.queue) + teller.count)
    time_points.append(env.now)
```

```
def setup(env, teller, lam, mu):
    """Generate customers with exponential interarrival times."""
    cid = 0
    while True:
        # Interarrival time ~ Exp(lam)
        interarrival = np.random.exponential(1 / lam)
        yield env.timeout(interarrival)
        cid += 1
        env.process(customer(env, cid, teller, mu))

# --- Parameters ---
lam = 1/ 3# arrival rate
mu = 1/ 5# service rate
# --- Simulation environment ---
env = simpy.Environment()
teller = simpy.Resource(env, capacity=1)
env.process(setup(env, teller, lam, mu))
env.run(until=50)
```

```
print("\nSystem size observations:")
print(system_size)
print("\nTime points:")
print(time_points)
```

Point Estimation

Sample Mean

$$\hat{\mu} = \overline{X} = \frac{1}{n} \sum_{i=1}^n X_i,$$

- $E[\overline{X}] = \mu$, so $\overline{X}$ is unbiased.
- Since X_i are independent and identically distributed, we have

$$\text{Var}(\overline{X}) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n},$$

Sample Variance

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \overline{X}_n)^2.$$

Confidence Intervals for the Mean

$$\frac{\overline{X} - \mu}{S/\sqrt{n}} \overset{d}{\rightarrow} \text{Normal}(0, 1).$$

Rearranging gives the approximate 100(1 − α)% confidence interval:

$$\mu \in \left(\overline{X} - z_{\alpha/2} \frac{S}{\sqrt{n}}, \overline{X} + z_{\alpha/2} \frac{S}{\sqrt{n}} \right).$$

where $z_{\alpha/2}$ is the standard-normal quantile (e.g. 1.96 for 95

Example

With $n = 40$, $\overline{X} = 4.8$, $S = 1.6$:

$$\text{CI} = 4.8 \pm 1.96 \frac{1.6}{\sqrt{40}} = (4.3, 5.3).$$

Precision-Driven Stopping Rules

A Method for Determining When to Stop Generating New Data

- Choose an acceptable value d for the standard deviation of the estimator.
 - Generate at least 100 data values.
 - Continue to generate additional data values, stopping when you have generated k values and $z_{\alpha/2} \frac{S_k}{\sqrt{k}} < d$, where S is the sample standard deviation based on those k values.
- That is,

$$k \geq \left(\frac{z_{\alpha/2} S_k}{d} \right)^2.$$

- The estimate of θ is given by

$$\bar{X} = \frac{1}{k} \sum_{i=1}^k X_i.$$

Recursive updating Storing all X_i is unnecessary.

$$\overline{X}_{n+1} = \overline{X}_n + \frac{X_{n+1} - \overline{X}_n}{n+1} \quad \text{with} \quad \bar{X}_0 = 0.$$

$$S_{n+1}^2 = \left(1 - \frac{1}{n}\right) S_n^2 + (n+1)(\overline{X}_{n+1} - \overline{X}_n)^2 \quad \text{with} \, S_1^2 = 0$$

Common Pitfalls

Pitfall	Consequence	Remedy
Correlated outputs (single long run)	CI too narrow	Independent replications.
Ignoring warm-up bias	Upward-biased estimates	Discard initial transient ("warm-up").
Too few replications (n < 30)	CLT unreliable	Require n ≥ 30.
Treating Bernoulli as continuous	Wrong variance formula	Use p(1 − p)/n.

```
def calculate_sample_size(sigma, error, confidence=0.95):
    alpha = 1- confidence
    z_score = norm.ppf(1 - alpha / 2)
    n = np.ceil((z_score * sigma / error) ** 2)
    return int(n)

# Calculate n for S = 4.77, E = 0.5
n = calculate_sample_size(sigma=4.77, error=0.5)
print(f'Required n (S = {std:.2f}, E = 0.5):', n)
```

```
# Try to fit exponential, gamma, log-normal and weibull
expon_params = st.expon.fit(data_clean, floc=0)
gamma_params = st.gamma.fit(data_clean, floc=0)
lognorm_params = st.lognorm.fit(data_clean, floc=0)
weibull_params = st.weibull_min.fit(data_clean, floc=0)
```

Variance Reduction

We want to estimate

$$I = E[h(U)], \quad U \sim \text{Uniform}(0, 1).$$

Crude estimator:

$$\hat{I}_C = \frac{1}{n} \sum_{i=1}^n h(U_i).$$

For each generated U , use its **Antithetic estimator** pair $1 - U$:

$$\hat{I}_A = \frac{1}{n/2} \sum_{i=1}^{n/2} \frac{h(U_i) + h(1 - U_i)}{2}.$$

Variance derivation Let $X = h(U)$ and $Y = h(1 - U)$

$$\text{Var}\left(\frac{X + Y}{2}\right) = \frac{1}{4}(\text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y)).$$

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

Example

$$I = \int_0^1 \sqrt{x} \, dx = \frac{2}{3}.$$

Crude estimator and $U \sim \text{Uniform}(0, 1)$

For n samples:

$$\hat{I}_C = \frac{1}{n} \sum_{i=1}^n U_i$$

Check unbiasedness:

$$\mathbb{E}[\sqrt{U}] = \int_0^1 \sqrt{x} \, dx = \frac{2}{3} = I.$$

Thus the crude estimator is unbiased.

Antithetic estimator and its unbiasedness

Antithetic variates use the negatively correlated pair $(U, 1 - U)$.

For n samples

$$\hat{I}_A = \frac{1}{n} \sum_{i=1}^n (U_i + (1 - U_i)).$$

Unbiasedness:

$$\mathbb{E}[\sqrt{U}] = \mathbb{E}[\sqrt{1 - U}] = \frac{2}{3}.$$

$$\mathbb{E}[\hat{I}_A] = \frac{1}{2} \left(\frac{2}{3} + \frac{2}{3} \right) = \frac{2}{3} = I.$$

Thus, the antithetic estimator is also unbiased.

Variance and Covariance Calculations

Variance

Since $U \sim \text{Uniform}(0, 1)$

$$E[\sqrt{U}] = \int_0^1 \sqrt{x} \, dx = \frac{2}{3}.$$

$$E[(\sqrt{U})^2] = E[U] = \int_0^1 x \, dx = \frac{1}{2}$$

$$\text{Var}(\sqrt{U}) = E[U] - (E[\sqrt{U}])^2 = \frac{1}{18} \quad \text{Var}(\hat{I}_A) = \frac{18\pi - 56}{288}$$

```
# --- Setup ---
N = 10000# number of crude samples
true_value = 2/3 # exact value of the integral
# --- Crude Monte Carlo ---
u_crude = np.random.uniform(0, 1, N)
y_crude = np.sqrt(u_crude)
est_crude = np.mean(y_crude)
var_crude = np.var(y_crude, ddof=1) / N
# --- Antithetic Variates ---
N_pairs = N // 2
u_source = np.random.uniform(0, 1, N_pairs)
# h(U) and h(1-U)
x1 = np.sqrt(u_source)
x2 = np.sqrt(1 - u_source)
y_antithetic = (x1 + x2) / 2
est_av = np.mean(y_antithetic)
var_av = np.var(y_antithetic, ddof=1) / N_pairs
# Variance Reduction Factor
variance_reduction = var_crude / var_av
```

Example

Passengers arrive according to a Poisson process with rate $\lambda = 10$ per hour = $\frac{1}{6}$ per minute, and service times are exponential with rate $\mu = 15$ per hour = $\frac{1}{4}$ per minute. Interarrival times:

$$A = -\frac{1}{\lambda} \ln U = -6 \ln U, \quad S = -\frac{1}{\mu} \ln U = -4 \ln U.$$